Quantifying Symmetries: How Optimisers Impact the Functional Dimension

Johanna Marie Gegenfurtner [*1] **Naima Elosegui Borras** [*1] **Georgios Arvanitidis** [1]

Abstract

Over-parametrised neural networks exhibit extensive symmetries in the parameter space. We investigate which optimisers are implicitly biased towards parameter solutions with a larger number of hidden symmetries. We study this in regression experiments by evaluating the functional dimension (the number of independent directions in the parameter space along which the network changes), which serves as an empirical tool to evaluate the prevalence of hidden symmetries. Moreover, we relate our results on functional dimension to flatness metrics involving the Hessian of the loss with respect to the parameters.

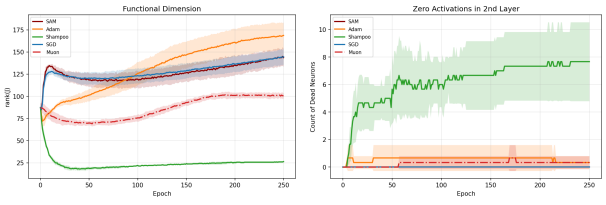


Figure 1. **Left:** We plot the development of the FD subject to different optimisers. Here, we used a three-layer ReLU network, using a teacher-student setup. Second order methods like Shampoo favour a low FD, indicating the prevalence of hidden symmetries. Additional plots can be found in Appendices D and E. **Right:** Hidden symmetries can, for example, arise through dead neurons. We count the number of dead neurons in the second layer.

1. Introduction

Neural networks owe much of their success to their remarkable expressivity. For a given architecture, a function is represented through its parameters; however, this representation is inherently redundant, as multiple parameter configurations can encode precisely the same function. These sets of equivalent parametrisations are referred to as *fibres*. For ReLU networks, characterising these fibres is generally challenging. We call a fibre trivial if it only exhibits the well-known global symmetries generated by neuron permutation and positive rescaling.

Beyond well-understood symmetries such as scaling and permutation, additional *hidden symmetries* may exist, though their prevalence is not yet fully understood beyond specific width and depth settings. Phuong and Lampert (2020) show that the fibres are trivial for architectures of non-increasing width, as are the fibres of generic parameters of shallow networks (Petzka et al., 2020). Recently, Ramakrishnan (2026) provided a complete classification for two-layer networks, describing a discrete sign-flip symmetry for non-generic parameters. Gegenfurtner et al. (2026) characterises the fibres of three-layer bottleneck architectures and introduces additional continuous symmetries. The latter work also indicates that complete symmetry classifications in more general parameter configurations might be unfeasibly hard.

The analytic hardness motivates the need for empirical methods to further understand hidden symmetries in deep ReLU networks. Grigsby et al. (2023) proposed the functional dimension (FD) as an empirical tool to quantify them, however, they only evaluated the FD at initialisation. We aim to extend their work by studying how the FD evolves throughout training, and how it can serve as a complexity measure, as already proposed by Bona-Pellissier et al. (2024).

Moreover, the choice of optimisation algorithm can affect the quality of learnt solutions (Pascanu et al., 2025). In classification settings, Fan et al. (2026) and Wang et al. (2021) show that adaptive optimizers with exponential moving averages on momentum converge to max-margin solutions. Huh (2020) demonstrate that the training trajectory of natural gradient descent (curvature corrected) exhibits accelerated convergence dynamics yet inferior generalisation, unless fractional corrections are applied. This raises the question of whether different optimisation algorithms favour particular representatives within a fibre, whether they tend to avoid or break symmetries, and how flat and balanced the solutions they converge to are.

[*]Equal contribution [1]Technical University of Denmark. Correspondence to: Johanna Marie Gegenfurtner <johge@dtu.dk>, Naima Elosegui Borras <nelbo@dtu.dk>.

Workshop on Weight-Space Symmetries, held in conjunction with the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. 2026.

Our repository can be found under the following link. https://github.com/naimaeb/sym_opt

1.1. Our contributions:

- We demonstrate empirically that from the tested optimisers, the only one taking second-order cross-parameter information into account (Shampoo) favours solutions with a lower FD, whereas optimisers that focus on first-order information favour higher FD parameter configurations. Furthermore, we find that particularly Adam favours parametrisations during training and at convergence with a lower FD as the number of training samples decreases.
- We provide a preliminary analysis of our empirical findings, investigating the optimisation algorithms and how they influence FD during training in ReLU networks in a student teacher task and a sine task.
- We further show how the FD is tied to flatness of the loss, establishing a measure of how balanced a minimum is.

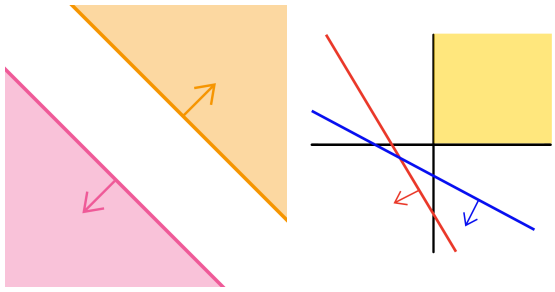


Figure 2. Mechanisms that induce hidden symmetries.
Left: The pink and orange line represent the hyperplanes induced by two neurons. The shaded areas are their active domains – since they do not intersect, the neurons are never active together.
Right: In yellow, we see the image of the first layer neurons, in red and blue the hyperplanes associated to the second layer activations. Since they do not intersect the positive orthant, and they are oriented away from it, their activations are zero. Hence, any reparametrisation preserving negative signs yields the same function.

2. Problem setup and definitions

Setting and notation We consider multi-layer perceptrons (MLPs) of L layers with ReLU activation functions. For a fixed architecture $\mathcal{A} = (n_0, \dots, n_L)$, a parameter is a tuple $\theta = (W^{(L)}, b^{(L)}, \dots, W^{(1)}, b^{(1)})$, where $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $b^{(\ell)} \in \mathbb{R}^{n_\ell}$, $\ell = 1, \dots, L$. We denote the parameter space by $\Theta_A = \prod_{\ell=d}^{\ell=1} \mathbb{R}^{n_\ell \times n_{\ell-1}} \times \mathbb{R}^{n_\ell}$. The chosen architecture now gives rise to a mapping $\theta \to f_\theta$ from the parameter space $\Theta_\mathcal{A}$ to the space of continuous piecewise affine-linear (CPWL) functions from $\mathbb{R}^d$ to $\mathbb{R}^D$, where $d = n_0, D = n_L$. In particular, let $f_\theta : \mathbb{R}^d \to \mathbb{R}^D$ be the layer wise composition

$$f_\theta(x) = W^{(L)}\sigma(W^{(L-1)}(\dots\sigma(W^{(1)}x+b^{(1)})+b^{(L-1)}))+b^{(L)},$$

with $\sigma(z) = \max(0, z)$. Let z_i^ℓ be the $i-$th preactivation of layer ℓ, and $\alpha_i^\ell = \sigma(z_i^\ell)$ the $i-$th neuron of layer ℓ. To evaluate which function fits a given data set $\mathcal{D} = \{x_i, y_i\}_{i=1,\dots,N}$ best, we minimise the empirical loss function $\mathcal{L} : \Theta \to \mathbb{R}$. In our problem setup we use the mean squared error (MSE) loss.

3. Hidden symmetries of ReLU networks

Hidden symmetries refer to parameter space symmetries different from positive scaling and neuron permutation. Positive measure subsets without hidden symmetries exist in various depth and width configurations (Phuong and Lampert, 2020; Rolnick and Kording, 2020). Otherwise, they may arise, for example, when two neurons i, m in layer ℓ are never active together (see Figure 2). This happens if for all $j = 1, \dots, n_{\ell-1}$ $W_{i,j}^{(\ell)} = -W_{m,j}^{(\ell)}, b_i^{(\ell)} \leq -b_m^{(\ell)}$. In the case of equality, their hyperplanes overlap, but have opposite orientation. Hidden symmetries may also arise from stably inactive neurons. Since they are empirically easiest to track, we will mostly focus on their prevalence.

Definition 3.1. A neuron $\alpha_i^{(\ell)}$ is inactive at an input $x \in \mathbb{R}^d$ if $\alpha_i^{(\ell)}(x) = 0$. For $\ell > 1$, a neuron $\alpha_i^{(\ell)}(x) = \sigma\left(W_i^{(\ell)}\alpha^{(\ell-1)}(x) + b_i^{(\ell)}\right)$ is called stably inactive if it is inactive for all $x \in \mathbb{R}^d$ under small perturbations of $W_i^{(\ell)}, b_i^{(\ell)}$.

It follows promptly that for $\ell \geq 2$, $\alpha_i^{(\ell)}$ is stably inactive if and only if $W_{i,j}^{(\ell)}, b_i^{(\ell)} < 0$ for all $j = 1, \dots, n_{\ell-1}$.

4. Functional Dimension (FD)

The FD refers to the number of independent directions in the parameter space that change the model function f_θ. Thus, a high FD indicates the absence of hidden symmetries locally, and therefore provides evidence toward the identifiability of the network.

Definition 4.1 (Grigsby et al. (2022))**.** Let $X = \{x_1, \dots, x_N\}$ be a set of parametrically smooth points[1] in $\mathbb{R}^d$, and let

$$J_X(\theta) = [\nabla_\theta f_\theta(x_n)]_{n=1\dots N}^T \in \mathbb{R}^{DN \times \mathbf{dim}(\Theta_\mathcal{A})} \quad (1)$$

be the stack of Jacobians of $f_\theta : \mathbb{R}^d \to \mathbb{R}^D$ evaluated at the points in X, vertically concatenating matrices of $D \times \mathbf{dim}(\Theta_\mathcal{A})$ for each n. We define the batch functional dimension of θ as $\mathbf{dim}_{\text{ba.fun}}(\theta, X) = \mathbf{rank}(J_X(\theta))$.

Taking the supremum over all possible finite subsets X,

[1] A point $x \in X$ is *parametrically smooth* if the gradient of the function with respect to θ is well defined at x. This is a realistic assumption, since the complement of the set of parametrically smooth points has measure zero for ReLU architectures.

we obtain the FD. Note that for a sufficiently large and spread out data set X, the batch functional dimension over X approximates the FD of a function.

Definition 4.2 (Grigsby et al. (2022))**.** The functional dimension (FD) of $\theta \in \Theta$ is given by

$$\mathbf{dim}_{\text{fun}}(\theta) = \sup_X \{\mathbf{rank}(J_X(\theta)) \mid X \subseteq \mathbb{R}^d\}, \tag{2}$$

where all X are finite sets of parametrically smooth points.

Lastly, we informally state two key facts, which motivate the study of the FD. This section is adapted from Grigsby et al. (2022).

- If $\mathbf{dim}_{\text{fun}}(\theta)$ does not attain its theoretical maximum bound $\left(\sum_{i=1}^{L} n_{i-1}n_i\right) + n_L$, then in a neighbourhood of θ, there are symmetries beyond scaling and permutation. Hence, a low $\mathbf{dim}_{\text{fun}}(\theta)$ serves as an indicator of hidden symmetries.
- $\mathbf{dim}_{\text{fun}}(\theta)$ is invariant under positive scaling and permutation (Bona-Pellissier et al., 2024), yet $\mathbf{dim}_{\text{fun}}(\theta)$ is not constant across the fibre: there may exist θ_1, θ_2 with $f_{\theta_1} = f_{\theta_2}$ (induced from non-trivial symmetries) but $\mathbf{dim}_{\text{fun}}(\theta_1) \neq \mathbf{dim}_{\text{fun}}(\theta_2)$. Thus, $\mathbf{dim}_{\text{fun}}(\theta)$ may be used as a complexity measure and to analyse implicit bias of solutions found by different learning algorithms.

5. Connections to sharpness

In this section, we will demonstrate that the FD is bounding several established measures of sharpness. Rather than measuring the sharpness or flatness of a minimum in the parameter space, we believe that FD serves as a complexity measure of a function. Specifically, we will demonstrate that together with the eigenvalues of the Hessian of the loss function, it allows us to better describe the shape of minima. Moreover, several Hessian based sharpness measures are not reparametrisation invariant (Dinh et al., 2017), whereas the FD is already accounting for continuous parameter space symmetries.

Since we are interested in the solution an optimiser ultimately finds, we evaluate several flatness measures at the end of training. We denote an interpolation solution by θ^*, meaning that f_{θ^*} fits the training points perfectly: for all $i = 1, \ldots, N$ we have $f_{\theta^*}(x_i) = y_i$. In the case of the MSE loss,

$$\nabla^2_\theta \mathcal{L}(\theta^*) \approx \frac{1}{N} J_X(\theta^*)^T J_X(\theta^*), \tag{3}$$

which is also known as the Gauss-Newton approximation (Dauphin et al., 2024). This implies that

$$\mathbf{trace} \nabla^2_\theta \mathcal{L}(\theta^*) \leq \mathbf{dim}_{\text{fun}}(\theta^*) \lambda_{max}, \tag{4}$$

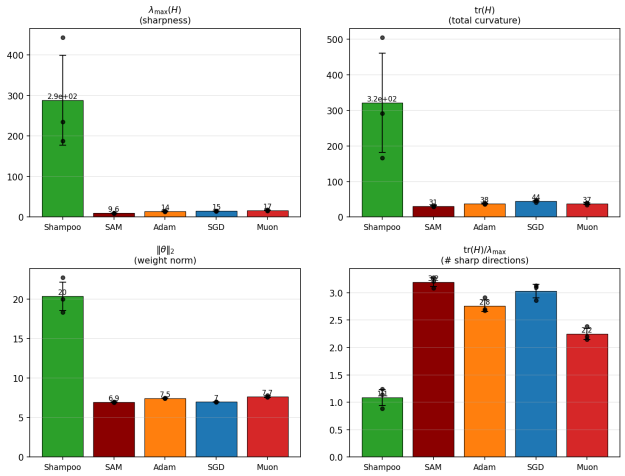


Figure 3. Sharpness metrics for 100 training points. We observe that the minima found for Shampoo have high eigenvalues of the loss Hessian, however there are fewer significant eigenvalues. In other words, the magnitude is more evenly spread for the minima found by other optimisers.

where λ_{max} is the largest eigenvalue of the Hessian $\nabla^2_\theta \mathcal{L}(\theta^*)$.

5.1. Scalar curvature of the loss

We now evaluate the scalar curvature of the loss manifold , which is the graph of the loss function:

$$\mathcal{M} = (\Theta, \mathcal{L}(\Theta)). \tag{5}$$

It can be equipped with the pull back metric

$$g(\theta) = I + \nabla_\theta \mathcal{L}^T \nabla_\theta \mathcal{L}, \tag{6}$$

which equips each point of $\mathcal{M}$ with a scalar product, and hence a way to measure distances on the manifold. Using the metric, we can also compute the scalar curvature of $\mathcal{M}$.

Theorem 5.1. *(Pouplin et al., 2023) The scalar curvature of $\mathcal{M}$ is given by*

$$\begin{aligned} K(\theta) &= \beta(\mathbf{trace}(\nabla^2_\theta \mathcal{L}(\theta))^2 - \mathbf{trace}(\nabla^2_\theta \mathcal{L}(\theta)^2)) \\ &+ 2\beta^2 (\nabla_\theta \mathcal{L}(\theta)^T (\nabla^2_\theta \mathcal{L}(\theta)^2 \\ &- \mathbf{trace}(\nabla^2_\theta \mathcal{L}(\theta)) \nabla^2_\theta \mathcal{L}(\theta)) \nabla_\theta \mathcal{L}(\theta)), \end{aligned} \tag{7}$$

where $\beta = \left(1 + ||\nabla_\theta \mathcal{L}(\theta)||^2\right)^{-1}$.

For an interpolation solution, it is easy to verify that $\beta(\theta^*) = 1$ and $\nabla_\theta \mathcal{L}(\theta^*) = 0$, and hence the second summand of $K(\theta^*)$ vanishes. We see that at such a minimum, the scalar curvature of the loss manifold is bounded by the FD. For more details and the proof of this results, we refer to Appendix A.

Theorem 5.2. *Let λ_{max} denote the largest eigenvalue of $\nabla^2_\theta \mathcal{L}(\theta)$, and $\mathbf{dim}_{ba.fun}(\theta, X) = \mathbf{rank}(J_X(\theta))$ be the batch*

FD of f_θ over the training points $X = \{x_i\}$. The scalar curvature $K(\theta^*)$ at an interpolation solution θ^* is bounded as follows:

$$0 \leq K(\theta^*) \leq (\mathbf{dim}_{ba.fun}(\theta^*, X) - 1) \cdot \mathbf{trace}\left(\nabla^2_\theta L(\theta^*)^2\right). \quad (8)$$

5.2. Effective rank

Let $G(\theta) = J_X(\theta)J_X(\theta)^T$ be the *empirical Neural Tangent Kernel* (NTK) (Jacot et al., 2018). We denote the singular values of $J_X(\theta)$ (which are the square roots of the eigenvalues of $G(\theta)$) by σ_k[2]. The effective rank of $J_X(\theta)$ is given by

$$\mathrm{er}(J_X(\theta)) = \exp\left(-\sum_k \frac{\sigma_k}{\|\sigma\|_1} \cdot \log\left(\frac{\sigma_k}{\|\sigma\|_1}\right)\right). \quad (9)$$

More generally, the effective rank is the exponential of the Shannon entropy, or equivalently the Rényi entropy of order $\alpha \to 1$ (Rényi, 1961; Roy and Vetterli, 2007). More details on this can be found in Appendix B.1. An easy computation in Appendix B.2 shows that

$$0 < \frac{\mathrm{er}(J_X(\theta))}{\mathbf{dim}_{\text{ba.fun}}(\theta, X)} \leq 1. \quad (10)$$

Moreover, the quotient $\frac{\mathrm{er}(J_X(\theta))}{\mathbf{dim}_{\text{ba.fun}}(\theta,X)}$ is low if there is one dominant singular value, and the inequality on the right holds with equality if and only if all positive singular values are equal. Thus, the ratio of effective rank to batch FD

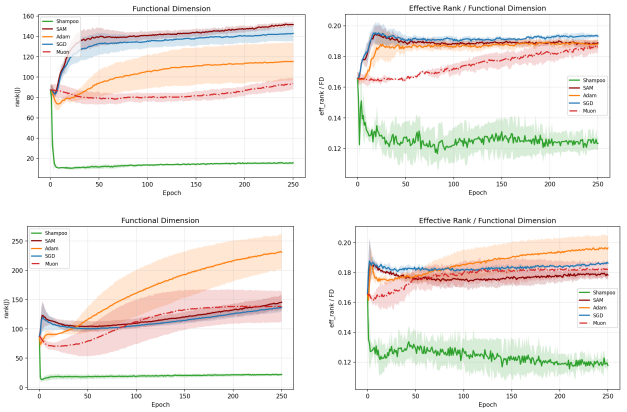


Figure 4. FD (**left**) and effective rank divided by FD (**right**). For 100 training points (**top**) and 1000 training points (**bottom**) Remarkably, ratio of effective rank to FD changes very little during training. Moreover, Adam (and to a smaller degree Muon) is particularly sensitive to the number of training points.

reveals how close the singular value distribution of $J_X(\theta)$ is to being uniform on its support, or equivalently, how uniformly the sensitivity of the network is distributed across the available independent parameter directions. Figure 4 shows that Shampoo has fewer steep directions, and hence is less balanced than the other optimisers.

[2]We note that the NTK matrix $G(\theta)$ and the GN matrix $J_X(\theta)^T J_X(\theta)$ from Equation 3 have the same non-zero eigenvalues, as they are Gram matrices of $J_X(\theta)$ and $J_X(\theta)^T$.

6. The impact of optimisers in student-teacher regression tasks

In this section we investigate the implicit bias of optimisers towards parameter configurations with higher or lower FD. In particular, we consider an over-parametrised student teacher regression task, where the teacher is a one-layer ReLU network with small isotropic Gaussian noise. Details can be found in Appendix C. Similar learning settings have been thoroughly studied in ReLU networks (Akiyama and Suzuki, 2021). We chose to focus on functions that we know the student is able to express, anticipating that the optimisers reach different representatives of the same fibre – however the latter is hard to verify in practice. For completeness, we also consider teacher functions that are not affine-linear, for example trigonometric functions. The results can be found in Appendix E. When evaluating the FD, we consider a non-zero singular value λ_k if $\lambda_k \geq \lambda_{max} \times 10^{-3}$ to not account for noise due to numerical imprecision.

Stochastic Gradient Descent (SGD). It was conjectured by Grigsby et al. (2023) that SGD lowers the FD of a network. Following Dherin et al. (2021), the gradient flow of SGD (for a scalar-valued function f and MSE loss) follows in expectation a regularised loss of the form

$$\mathcal{L}_{SGD}(\theta) \approx \mathcal{L}(\theta) + \frac{\eta}{4|\mathcal{D}|} \sum_{x,y \in \mathcal{D}} (f_\theta(x) - y)^2 \|\nabla_\theta f_\theta(x)\|^2 \quad (11)$$

with learning rate η. Hence, especially when the error is large, as in early stages of the training, $\|\nabla_\theta f_\theta(x)\|^2$ is regularised. We argue that this, however, does not necessarily regularise the FD. Indeed, $\|\nabla_\theta f_\theta(x)\|^2$ may be small for all $x \in \mathcal{D}$, but since the rank of a matrix is invariant to scaling, we may still have a high FD. In fact SGD exhibits a high FD, and a large number of relatively sharp directions in the loss landscape in our experiments (see Figure 3).

Sharpness-Aware Minimisation (SAM), introduced by Foret et al. (2020), specifically aims to reduce the sharpness of the loss, aiming for better generalisation. Andriushchenko et al. (2023), who investigated feature ranks, prove that for two-layer ReLU networks, SAM increases the number of inactive neurons, lowering the FD. However, they investigate data-dependent inactivity, and our definition of stably inactive neurons requires positive inputs, or at least three layers. We show that their theoretical guarantees do not straightforwardly extend to three layers in Appendix F.1. Note that SAM is used with a base optimiser, in our

experiments we use SGD. Further, empirically we did not observe particularly many dead neurons compared to the other optimisers.

We further consider **Adam** (Kingma and Ba, 2014), **Muon** (Jordan et al., 2024; Bernstein, 2025), and **Shampoo** (Gupta et al., 2018) to explore how second-order gradient information and momentum updating affect FD. They differ in (1) what gradient information they carry, (2) how they update momentum, and (3) how they precondition gradient updates. Consider the update

$$\theta_{t+1} = \theta_t - \eta P_t^{-1} g_t, \tag{12}$$

where g_t is a gradient estimate and P_t a preconditioner.

1. First order methods only use gradients $\nabla_\theta \mathcal{L}(\theta)$, whereas second order methods use Hessians $\nabla^2_\theta \mathcal{L}(\theta)$ or curvature information. While Adam and Shampoo are strictly speaking first order methods, they use second-order statistics of gradients as a proxy for curvature. Muon is strictly first-order.

2. Adam adapts the learning rate based on the first and second moments of the gradients $\nabla_\theta \mathcal{L}(\theta)$, speeding up convergence. This is done by using an exponential moving average (EMA), keeping track of previous gradients. The momentum is updated by

$$M_t = \beta_1 M_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L}(\theta_t), \tag{13}$$

   for $\beta_1 \in [0, 1)$. This equips the optimiser with previous gradient information, speeding up convergence and dampening large oscillations. Similarly to Adam, Muon keeps an EMA of the gradient terms, however it maintains the structure of M_t and normalises the singular values of the gradient matrix, forcing directions with small magnitudes to be equally explored. We consider Shampoo without momentum.

3. Adam uses a diagonal preconditioner of elementwise squared gradients. In contrast, Shampoo uses matrix-valued left and right preconditioners constructed from gradient Gram matrices, capturing correlations between parameters. As a result, Shampoo incorporates gradient covariance information in its off-diagonal structure. Both use an EMA for the preconditioners. For further discussion and related work to implicit bias of optimisers, see Appendix C.2.

6.1. Results

Shampoo finds minima with low FD. We find that Shampoo has low FD, partly due to the high number of dead neurons. At the same time, Shampoo exhibits higher value for several sharpness measures, and a larger parameter norm. This indicates that Shampoo selects a narrower, low-dimensional solution manifold, or equivalently, the trivial embeddings of subnetworks into a larger architecture. These findings are also reflected in Figure 4. We conjecture that this is due to the off-diagonal second-order information, as the other optimisers ignore these statistics. Future theoretical studies will investigate how gradient covariance affects FD. Optimisers such as SAM and SGD, which are said to be biased towards flat minima, exhibited rather high FD in our experiments. We believe that these minima distribute weights more evenly over all parameters, also leading to a lower weight norm, as also seen from the flatness measures in Figure 3 and Appendix D. While SAM, SGD and Muon do not reach the FD upper bound, they do not have a significant number of zero activations, suggesting the prevalence of other hidden symmetries.

Adam is sensitive to training set size. Adam increases FD as the training set increases, meaning when we consider a less extremely over-parametrised regime. One possible explanation for the observed increase in FD is that Adam responds to the additional constraints introduced by larger training sets by utilising a larger effective parameter subspace. The increase in FD with training set size is either significantly slower, as in Muon, or stalled as in SGD, SAM and Shampoo (see Appendices D, E). Future work will explore why Adam is more affected than other optimisers.

7. Conclusion and outlook

Regularisation mechanisms. Future work will focus on mathematically rigorously extending our empirical finding about optimiser effects on FD. Further, we want to study the impact of training set size. Here, we see a connection to the *lottery ticket hypothesis*, established by Frankle and Carbin (2018): suggesting that for over-parametrised networks, a subnetwork is chosen at initialisation and optimised, whereas the remaining neurons and their parameters may not impact the network output, thus leading to a lower FD. However, as shown by Grillo and Montúfar (2026), minimal parameters can still have redundancies. For a modern perspective on the lottery ticket hypothesis and network capacity, we refer to Pinson (2026).

Convergence, robustness, and generalisation. When the FD is low, optimisation is effectively constrained to a lower-dimensional subspace of the parameter space. On the other hand, from a point with a high FD, we have more flexibility. Thus, this should impact speed of convergence and robustness, and possibly lead to generalisation bounds related to the FD. Lastly, we plan on systematically exploring the effect of variables such as network width, momentum EMA weight and learning rate schedules.

Acknowledgements We warmly thank Kathryn Lindsey for numerous insightful discussions, valuable advice, and thoughtful proofreading. JMG, NEB, and GA have been supported by the DFF Sapere Aude Starting Grant "GADL". JMG would like to thank *Women in Machine Learning* for supporting her ICML registration. We are also grateful to all reviewers for their helpful and constructive feedback.

References

Shunta Akiyama and Taiji Suzuki. On learnability via gradient method for two-layer relu neural networks in teacher-student setting. In *International Conference on Machine Learning*, 2021. URL https://api.semanticscholar.org/CorpusID:235417345.

Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998. doi: 10.1162/089976698300017746.

Shun-ichi Amari, Jimmy Ba, Roger Baker Grosse, Xuechen Li, Atsushi Nitanda, Taiji Suzuki, Denny Wu, and Ji Xu. When does preconditioning help or hurt generalization? In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=S724o4_WB3.

Maksym Andriushchenko, Dara Bahri, Hossein Mobahi, and Nicolas Flammarion. Sharpness-aware minimization leads to low-rank features. *Advances in Neural Information Processing Systems*, 36:47032–47051, 2023.

Jeremy Bernstein. Deriving muon, 2025. URL https://jeremybernste.in/writing/deriving-muon.

Joachim Bona-Pellissier, François Malgouyres, and François Bachoc. Geometry-induced implicit regularization in deep relu neural networks. 2024.

Yann N Dauphin, Atish Agarwala, and Hossein Mobahi. Neglected hessian component explains mysteries in sharpness regularization. *Advances in Neural Information Processing Systems*, 37:131920–131945, 2024.

Benoit Dherin, Michael Munn, and David G. T. Barrett. The geometric occam's razor implicit in deep learning, 2021. URL https://arxiv.org/abs/2111.15090.

Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp minima can generalize for deep nets. In *International Conference on Machine Learning*, pages 1019–1028. PMLR, 2017.

Chen Fan, Mark Schmidt, and Christos Thrampoulidis. Implicit bias of spectral descent and muon on multiclass separable data. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=Zn2ajV1kTQ.

Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *arXiv preprint arXiv:2010.01412*, 2020.

Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.

Johanna Marie Gegenfurtner, Moritz Grillo, and Guido Montúfar. The symmetries of three-layer relu networks, 2026. URL https://arxiv.org/abs/2605.18319.

Elisenda Grigsby, Kathryn Lindsey, and David Rolnick. Hidden symmetries of ReLU networks. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 11734–11760. PMLR, 23–29 Jul 2023.

Julia Elisenda Grigsby, Kathryn A. Lindsey, Robert Meyerhoff, and Chen-Chun Wu. Functional dimension of feedforward relu neural networks. *ArXiv*, abs/2209.04036, 2022. URL https://api.semanticscholar.org/CorpusID:252185455.

Moritz Grillo and Guido Montúfar. Most relu networks admit identifiable parameters, 2026. URL https://arxiv.org/abs/2605.03601.

Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1842–1850, Stockholm, Sweden, 10–15 Jul 2018. PMLR. URL https://proceedings.mlr.press/v80/gupta18a.html.

Bobby He and Mete Ozay. Exploring the gap between collapsed amp; whitened features in self-supervised learning. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 8613–8634. PMLR, 17–23 Jul 2022. URL https://proceedings.mlr.press/v162/he22c.html.

Dongsung Huh. Curvature-corrected learning dynamics in deep neural networks. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 4552–4560. PMLR, 4 2020. URL https://proceedings.mlr.press/v119/huh20a.html.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014. URL https://api.semanticscholar.org/CorpusID:6628106.

Mikhail Kuznetsov, Ivan Butakov, Marina Munkhoeva, Alexey Frolov, and Ivan Oseledets. Information-theoretic unsupervised embedding quality evaluation. In *ICLR 2026 Workshop on Representational Alignment (Re^4-Align)*, 2026. URL https://openreview.net/forum?id=EwaNoLlEXT.

John M. Lee. *Introduction to Smooth Manifolds*. 2000.

Wu Lin, Felix Dangel, Runa Eschenhagen, Juhan Bae, Richard E. Turner, and Alireza Makhzani. Can we remove the square-root in adaptive gradient methods? A second-order perspective. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 29949–29973. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/lin24e.html.

Razvan Pascanu, Clare Lyle, Ionut-Vlad Modoranu, Naima Elosegui Borras, Dan Alistarh, Petar Velickovic, Sarath Chandar, Soham De, and James Martens. Optimizers qualitatively alter solutions and we should leverage this. 7 2025.

Henning Petzka, Martin Trimmel, and Cristian Sminchisescu. Notes on the symmetries of 2-layer relu-networks. In *Proceedings of the northern lights deep learning workshop*, volume 1, pages 6–6, 2020.

Mary Phuong and Christoph H Lampert. Functional vs. parametric equivalence of relu networks. In *International conference on learning representations*, 2020.

Hannah Pinson. It's not a lottery, it's a race: Understanding how gradient descent adapts the network's capacity to the task. *arXiv preprint arXiv:2602.04832*, 2026.

Alison Pouplin, Hrittik Roy, Sidak Pal Singh, and Georgios Arvanitidis. On the curvature of the loss landscape, 2023. URL https://arxiv.org/abs/2307.04719.

Pranavkrishnan Ramakrishnan. A complete symmetry classification of shallow relu networks. *arXiv preprint arXiv:2604.14037*, 2026.

Alfréd Rényi. On measures of entropy and information. In *Proceedings of the fourth Berkeley symposium on mathematical statistics and probability, volume 1: contributions to the theory of statistics*, volume 4, pages 547–562. University of California Press, 1961.

David Rolnick and Konrad Kording. Reverse-engineering deep relu networks. In *International conference on machine learning*, pages 8178–8187. PMLR, 2020.

Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *2007 15th European signal processing conference*, pages 606–610. IEEE, 2007.

Bohan Wang, Qi Meng, Wei Chen, and Tie-Yan Liu. The implicit bias for adaptive optimization algorithms on homogeneous neural networks. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10849–10858. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/wang21q.html.

A. The curvature of the loss manifold

For a more comprehensive introduction to manifolds, their curvature and other notions from differential geometry, we refer the interested reader to the classic textbook (Lee, 2000).

The FD bounds the scalar curvature, as shown in Theorem 5.2. The proof of which is given below.

Proof. From Theorem 5.1, we have that

$$\begin{aligned} K(\theta^*) &= \textbf{trace}(\nabla^2_\theta \mathcal{L}(\theta))^2 - \textbf{trace}(\nabla^2_\theta \mathcal{L}(\theta)^2) \\ &= \left(\sum_i \lambda_i\right)^2 - \sum_i \lambda_i^2 \\ &\leq \textbf{dim}_{\text{ba.fun}}(\theta, X) \sum_i \lambda_i^2 - \sum_i \lambda_i^2 \\ &= (\textbf{dim}_{\text{ba.fun}}(\theta, X) - 1) \cdot \sum_i \lambda_i^2. \end{aligned}$$

Here we leveraged the Cauchy-Bunyakovski-Schwarz inequality, which implies that

$$\left(\sum_i \lambda_i\right)^2 \leq \textbf{dim}_{\text{ba.fun}}(\theta, X) \cdot \sum_i \lambda_i^2.$$

□

This shows that both the FD and the eigenvalues of the Hessian bound the scalar curvature.

B. Details on the effective rank

B.1. The effective rank in the context of Rényi entropy.

If we consider the exponential of the Rényi entropy such that $\text{er}(J_X(\theta))_\alpha = \exp\left(\frac{1}{1-\alpha}\log\left(\sum_k p_k^\alpha\right)\right)$ we recover for $\alpha \to 0$ the numerical rank and for $\alpha \to \infty$ the NESum (He and Ozay, 2022; Kuznetsov et al., 2026), which in turn corresponds to the stable rank of $G(\theta)$ [3]. Increasing α progressively places more weight on the dominating eigenvalues; hence $\alpha \to 1$ represents a natural middle ground between counting all active directions equally (numerical rank), which requires setting a threshold for singular value detection, and focusing primarily on the dominant ones (NESum, stable rank). Moreover, in over-parametrised settings where there can be large number of zero singular values, choosing a quantity such as the condition number ($\frac{\sigma_{max}}{\sigma_{min}}$) will be non-informative, as $\sigma_{min} = 0$ in most cases we observe.

B.2. Computations

We make a few observations. $G(\theta)$ is positive semi-definite, hence all eigenvalues are non negative. Further,

$$\|\sigma\|_2^2 = \text{trace}(G(\theta)) = \|J_X(\theta)\|_F^2.$$

As shown in Roy and Vetterli (2007),

$$1 \leq \text{er}(J_X(\theta)) \leq \textbf{rank}(J_X(\theta)),$$

and thus

$$0 < \frac{\text{er}(J_X(\theta))}{\textbf{dim}_{\text{ba.fun}}(\theta, X)} \leq 1.$$

[3] As noted above, the eigenvalues of $G(\theta)$ are the squares of the singular values of $J_X(\theta)$ and NESum is given by $\frac{\sum_k \sigma_k}{\sigma_{max}}$ and stable rank of of $G(\theta)$ is given by $\frac{\sum_k \sigma_k^2}{\sigma_{max}^2}$

C. Implementation details and algorithms

The interested reader can find an anonymised repository under the following link: https://anonymous.4open.science/r/sym_opt-040F/README.md. It contains detailed instructions and comments, but we summarise some details in this section.

In our empirical studies, we started using evaluation sets X of order $100 \cdot \mathcal{D}_{\mathcal{A}}$ for smaller network width (≈ 6 neurons), to make sure that the set of evaluation points was not a limiting factor. Nevertheless, as we observed that sets of the order of $\frac{\mathcal{D}_{\mathcal{A}}}{10}$ were sufficient in the over-parametrised setting, we maintained these values empirically for computational efficiency. The student networks shown in our figures (unless specified otherwise) are three-layer ReLU networks, with $d = 10, n_1 = n_2 = 64, D = 1$, however this can be easily adapted. We summarise this in the following table.

Table 1. ReLUNet Architecture and Dataset Hyperparameters

Hyperparameter	**Value**
Input Dimension (D_{IN})	10
Hidden Layer 1 Size	64
Hidden Layer 2 Size	64
Output Dimension	1
Total Parameters	**4,929**
Training Samples (N_{TRAIN})	[100, 400 or 1000]
Test Samples (N_{TEST})	800
Samples to estimate fd	800
Target Noise (σ)	0.1

C.1. Tasks

We used the following teacher functions.

One-layer ReLU. The teacher function generates its output according to $y = \text{ReLU}(XW^*) + \epsilon$ where $\text{rank}(W*) = k$ (in our experiments $k = 5$) and $\epsilon \sim \mathcal{N}(0,\ 0.1)$ and $X \sim \mathcal{N}(0,\ I_d)$.

Trigonometric functions. The teacher network has outputs $y = \sin(XW_{proj}) + \epsilon$, where W_{proj} is a projection matrix onto 1D and $\epsilon \sim \mathcal{N}(0,\ 0.1)$

C.2. Optimisers and Algorithms

In this section we outline the optimisers we consider in our empirical studies, and provide further insights. A short summary of the hyperparameters is provided in the following table.

Table 2. Training Configuration

Parameter	**Setting**
Batch Size	64
Total Epochs	250
Loss Function	MSE

Table 3. Optimiser Configurations and Learning Rate Schedule

Optimiser	**Learning Rate**	**Weight Decay**	**Additional Hyperparameters / Notes**
SGD	1×10^{-2}	0.0	Momentum: 0.9
Adam	1×10^{-3}	0.0	Default coefficients ($\beta_1 = 0.9, \beta_2 = 0.99$)
Shampoo	1×10^{-1}	0.0	Kronecker-factored preconditioner
Muon	1×10^{-3}	0.0	Applied to 2D params; Adam for others
SAM	1×10^{-2}	0.0	Base: SGD, $\rho = 0.05$, Momentum: 0.9

Stochastic Gradient Descent. Instead of using the whole training set (as in gradient descent) when updating the parameters, SGD only uses a subset when evaluating the gradient of the loss function. This lowers the computational cost.

Algorithm 1 SGD with Momentum

Input: Learning rate η, momentum μ, initial parameters θ_0, velocity $v_0 = 0$
for $t = 0, 1, \dots$ **do**
  $g_t \leftarrow \nabla L(\theta_t)$; // Compute gradient
  $v_{t+1} \leftarrow \mu v_t + g_t$; // Update velocity
  $\theta_{t+1} \leftarrow \theta_t - \eta v_{t+1}$; // Update weights
end

Sharpness-Aware Minimisation. SAM aims to find minima in areas of the parameter space with a uniformly low loss, rather than isolated low loss points. This is intended to improve generalisation. Foret et al. (2020) formulate this as a min-max problem:

$$\min_{\theta} \max_{\|\epsilon\|_p \leq \rho} \mathcal{L}(\theta + \epsilon),$$

where $\|\cdot\|_p$ is the L^p-norm and the hyperparameter ρ is the radius of the L^p-ball around θ where we search for the highest loss. In other words, we aim to find a parameter θ^* such that in a ball of radius ρ the loss is uniformly low. To keep computational cost low, $\epsilon(\theta)$ is usually approximated by

$$\epsilon(\theta) = \rho \frac{\nabla_\theta \mathcal{L}(\theta)}{\|\nabla_\theta \mathcal{L}(\theta)\|_2}.$$

Algorithm 2 SAM

Input: Loss function L, parameters w_t, learning rate η, neighbourhood size ρ
$g_t \leftarrow \nabla_w L(w_t)$ $\hat{\epsilon}_t \leftarrow \rho \frac{g_t}{\|g_t\|_2}$; // Optimal perturbation
$g_t^{SAM} \leftarrow \nabla_w L(w_t + \hat{\epsilon}_t)$; // Gradient at "sharp" point
$w_{t+1} \leftarrow w_t - \eta g_t^{SAM}$; // Update weights

Adam. Adam uses an EMA of the diagonal of $\sqrt{\nabla_\theta \mathcal{L}(\theta) \nabla_\theta \mathcal{L}(\theta)^T}$ and of previous $\nabla \mathcal{L}(\theta)$ to adapt its learning rate. Wang et al. (2021) study implicit biases of adaptive optimisers, such as Adam, in homogeneous networks, showing that optimisers that use an exponential moving averages converge to max-margin solutions. Moreover, (Amari et al., 2021) study the implicit bias of preconditioning gradients, as done in Natural Gradient Descent (Amari, 1998), Shampoo or Adam (using the diagonal of the empirical Fisher information matrix) from the angle label noise and signal misalignment.

Algorithm 3 Adam

Input: Step size η, decay rates β_1, β_2, params θ_0, $m_0 = 0, v_0 = 0$
for $t = 1, 2, \dots$ **do**
  $g_t \leftarrow \nabla L(\theta_{t-1})$ $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$; // 1st moment
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$; // 2nd moment
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$; // Bias correction
  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$
end

Shampoo Shampoo multiplies the current matrix by right and left EMA of a Kronecker product of $\sqrt{\nabla_\theta \mathcal{L}(\theta) \nabla_\theta \mathcal{L}(\theta)^T}$. For more details on the relationship between these algorithms and the role of the square-root and generalisation see (Lin et al.,

2024). Note that of the optimisers we have studied, Shampoo is the only one that carries gradient covariance information, meaning one considers the off-diagonal entries of $\nabla_\theta \mathcal{L}(\theta) \nabla_\theta \mathcal{L}(\theta)^T$.

Algorithm 4 Shampoo (Gupta et al., 2018)

Input: Learning rate η, epsilon ϵ, initial weights $W_0 \in \mathbb{R}^{m \times n}$
Initialise $L_0 = \epsilon I_{m \times m}, R_0 = \epsilon I_{n \times n}$ **for** $t = 1, 2, \dots$ **do**
  $G_t \leftarrow \nabla_W L(W_{t-1})$ $L_t \leftarrow L_{t-1} + G_t G_t^\top$; // Left statistics
  $R_t \leftarrow R_{t-1} + G_t^\top G_t$; // Right statistics
  $H_L \leftarrow L_t^{-1/4}, \quad H_R \leftarrow R_t^{-1/4}$; // Preconditioners
  $W_t \leftarrow W_{t-1} - \eta (H_L \cdot G_t \cdot H_R)$
end

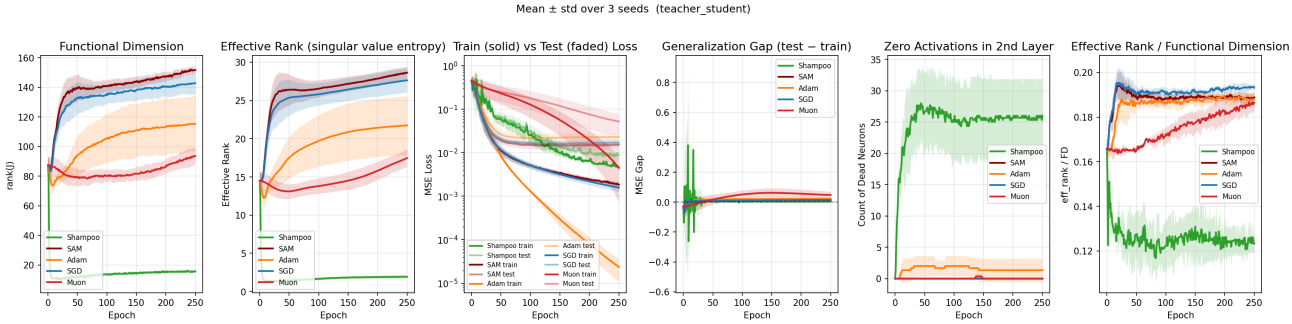
Muon, as described by (Jordan et al., 2024) and (Bernstein, 2025) is a first order optimiser designed for 2D matrices. It performs steepest descent with respect to the spectral norm by orthogonalizing the momentum matrix M as detailed in the algorithm below. This can be thought of as orthogonalizing $\nabla_\theta \mathcal{L}(\theta)$, it does so using the Newton-Schultz algorithm, avoiding the computational overhead of Singular Value Decomposition (SVD). This normalises the singular values of the gradient matrix, forcing directions with originally small magnitudes to be equally explored. (Fan et al., 2026) recently explored the implicit bias of Muon, showing that they converge to max-margin solutions (optimal classifier that maximises the distance between class data and decision boundary) measured by the spectral norm.

Algorithm 5 Muon (Bernstein, 2025)

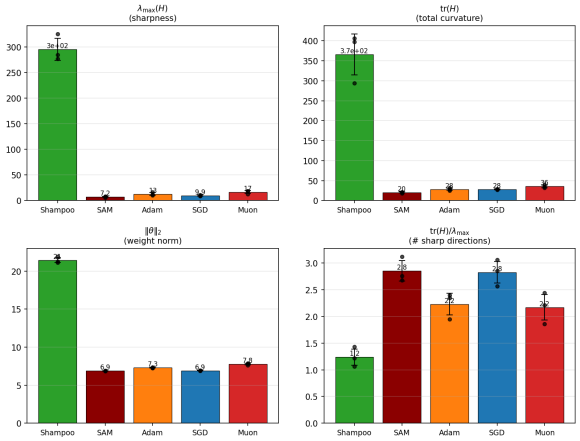
Input: Learning rate η, momentum μ
Initialize $M_0 = 0$ **for** $t = 1, 2, \dots$ **do**
  $G_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$; // Compute gradient
  $M_t \leftarrow \mu M_{t-1} + G_t$; // Update momentum
  $O_t \leftarrow \text{NewtonSchulz5}(M_t)$; // Orthogonalize momentum matrix
  $\theta_t \leftarrow \theta_{t-1} - \eta O_t$; // Update parameters
end
return θ_t

D. Full results for the ReLU student-teacher tasks

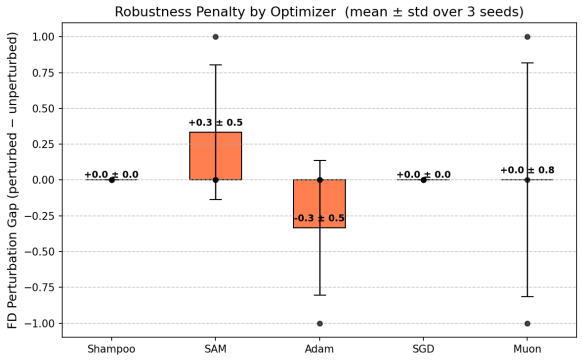
Training with N=100 points.



(a) Training dynamics and FD



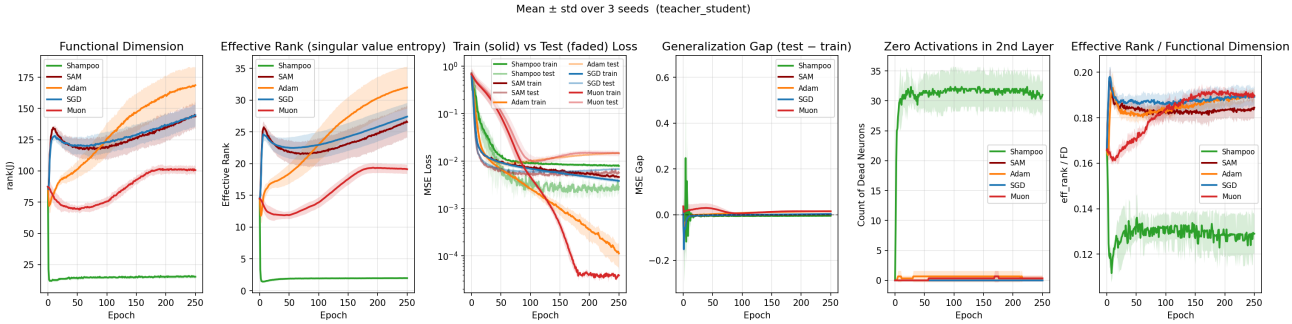
(b) Flatness metrics at the end of training



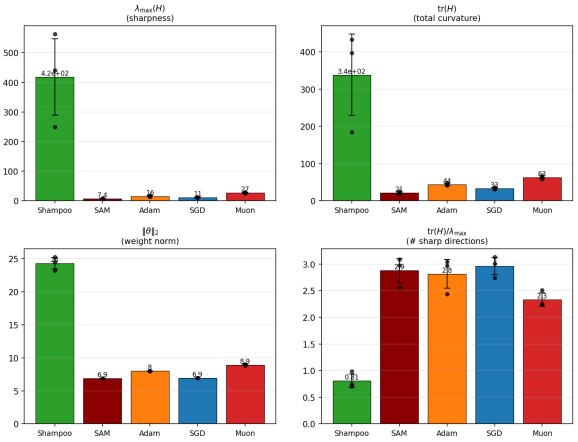
(c) Robustness to perturbations in the parameters and its effects on FD at the end of training

Figure 5. ReLU task for 100 training points and 800 test points

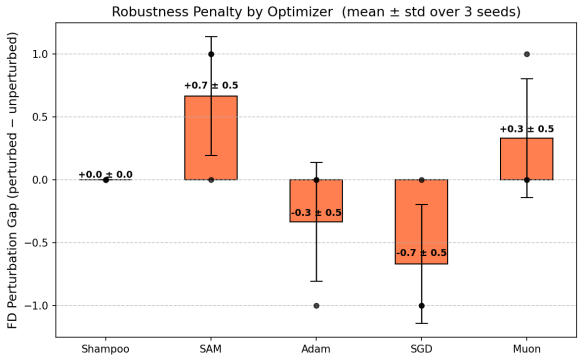
Training with N=400 points.



(a) Training dynamics and FD



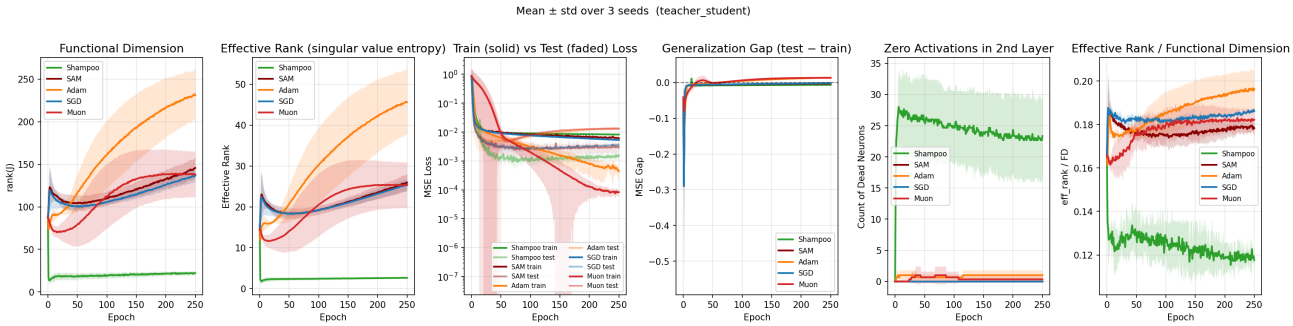
(b) Flatness metrics at the end of training



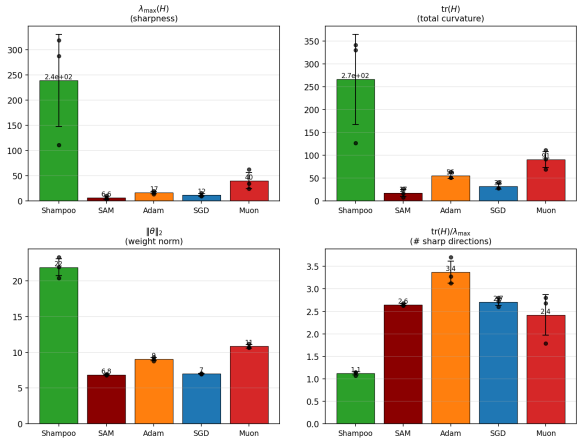
(c) Robustness to perturbations in the parameters and its effects on FD at the end of training

Figure 6. ReLU task for 400 training points and 800 test points

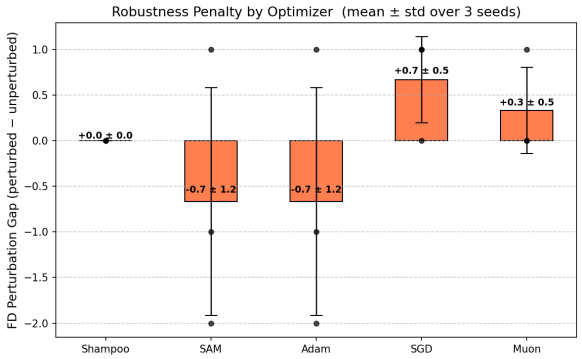
Training with N=1000 points.



(a) Training dynamics and FD



(b) Flatness metrics at the end of training



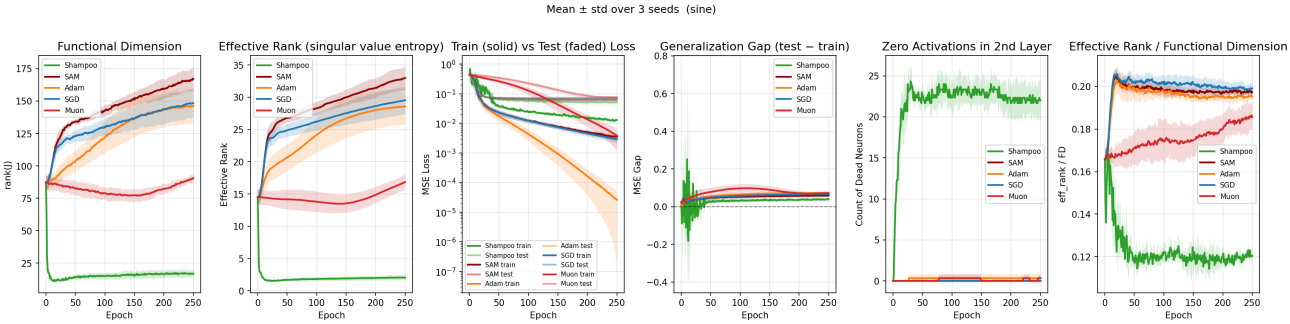
(c) Robustness to perturbations in the parameters and its effects on FD at the end of training

Figure 7. ReLU task for 1000 training points and 800 test points

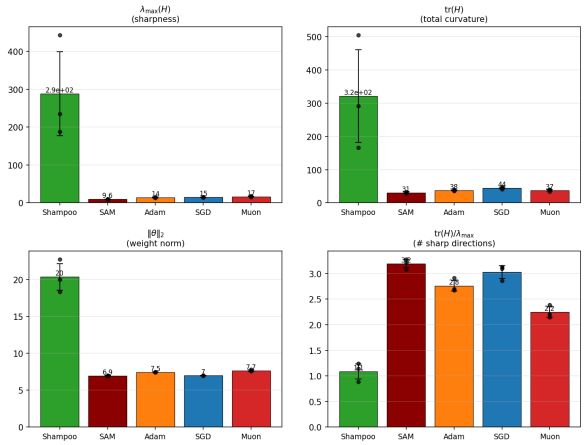
E. Results on the sine task

We now consider a task where the learnt network cannot perfectly replicate the teacher function, such as in the case of $y = \sin(XW_{proj}) + \epsilon$ as described in Section 6. We provide our empirical results for increasing numbers $(100, 400, 1000)$ of training points, maintaining two ReLU hidden layers of size 64, input dimension equal to 10 and output dimension equal to 1. The experiments were run for 3 different seeds.

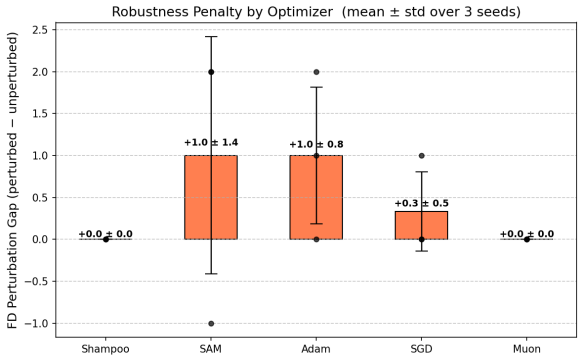
Training with N=100 points.



(a) Training dynamics and FD



(b) Flatness metrics at the end of training

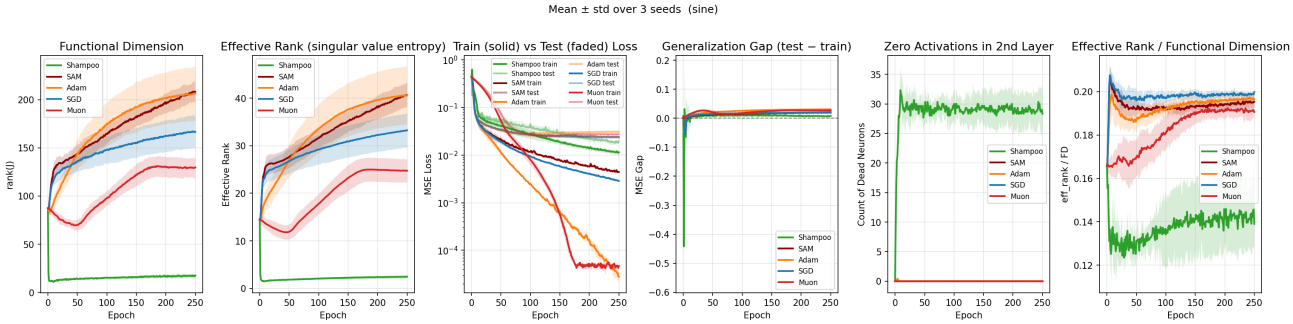


(c) Robustness to perturbations in the parameters and its effects on FD at the end of training

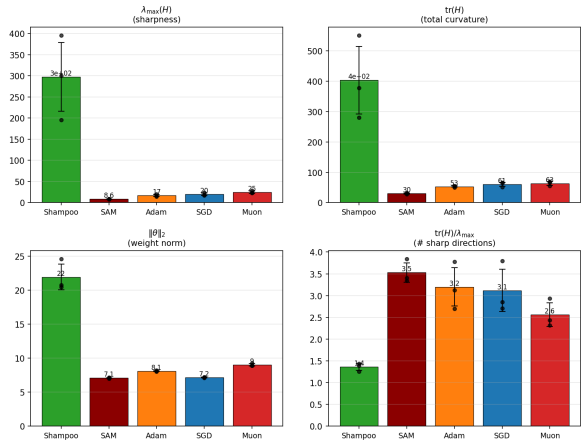
Figure 8. Sine task for 100 training points and 800 test points

Overall, these findings resemble the ReLU task.

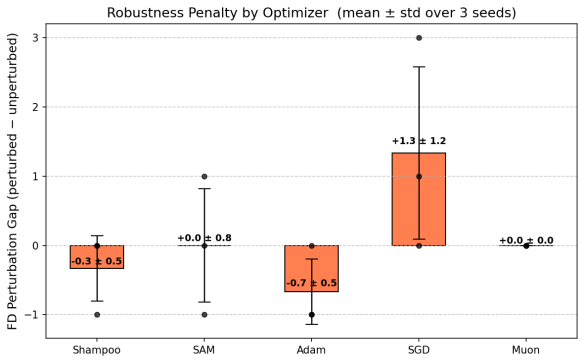
Training with N=400 points.



(a) Training dynamics and FD



(b) Flatness metrics at the end of training

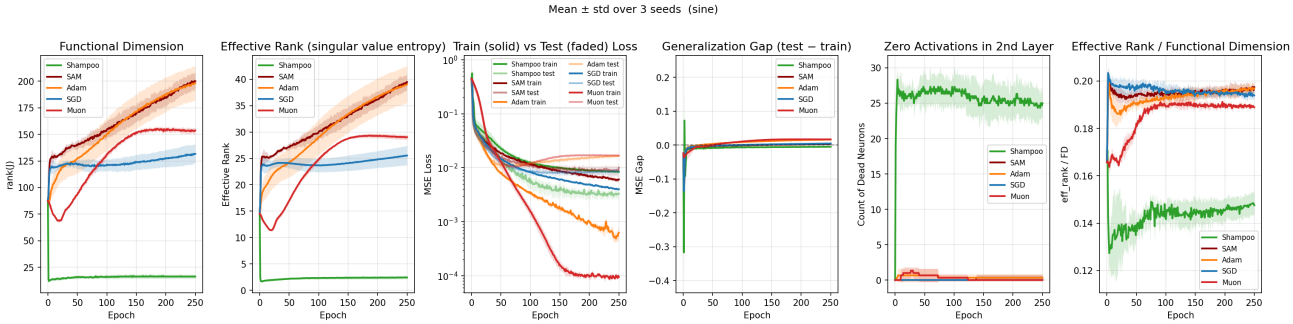


(c) Robustness to perturbations in the parameters and its effects on FD at the end of training

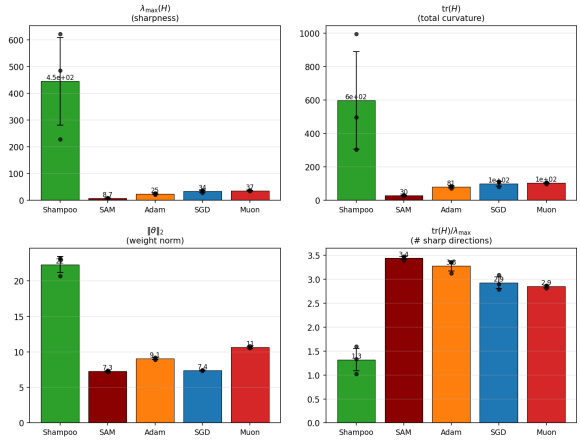
Figure 9. Sine task for 400 training points and 800 test points

Increasing the number of training points drastically changes the behaviour for Adam, which now exhibits a higher FD.

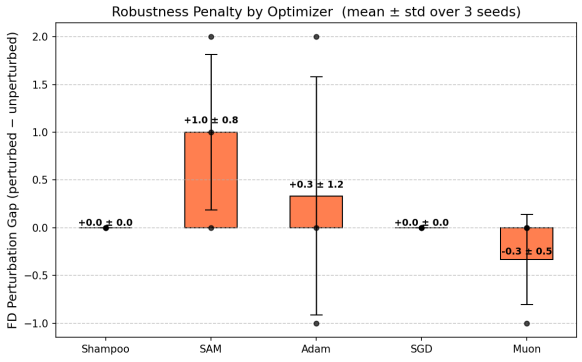
Training with N=1000 points.



(a) Training dynamics and FD



(b) Flatness metrics at the end of training



(c) Robustness to perturbations in the parameters and its effects on FD at the end of training

Figure 10. Sine task for 1000 training points and 800 test points

For 1000 training points, Adam and SAM have high FD.

F. Do implicit regularisers induce zero activations for three-layer ReLU networks?

Inspired by Andriushchenko et al. (2023), who consider two-layer networks, we want to investigate if implicit regularisers drive the weights toward zero activations. As shown previously, we need $W^{(2)}_{rk}, b^2_r \leq 0$ for all k for the neuron α^2_r to be stably inactive. Thus, we aim to show that at each step, we subtract a non-negative contribution from $W^{(2)}_{rk}, b^2_r$.

F.1. Computation for SAM

We consider the easiest case, $D = 1$. At each optimiser step $t \to t+1$ for SAM, we have that

$$\theta_{t+1} = \theta_t + \eta \nabla_\theta \left(\mathcal{L}(\theta) + \eta \|\nabla_\theta \mathcal{L}(\theta)\| + \mathcal{O}(\eta^2) \right).$$

Thus, we need to investigate the term $\nabla_\theta \|\nabla_\theta \mathcal{L}(\theta)\| = \frac{1}{2\|\nabla_\theta \mathcal{L}(\theta)\|} \nabla_\theta \|\nabla_\theta \mathcal{L}(\theta)\|^2$. Since we want to know how $W^{(2)}_{rk}$ is impacted, we evaluate

$$\nabla_{W^{(2)}_{rk}} \|\nabla_\theta \mathcal{L}(\theta)\|^2 = \frac{2}{N^2} \sum_{ij} \left(\nabla_{W^{(2)}_{rk}} f(x_i) \epsilon(x_i) \langle \nabla_\theta f(x_i), \nabla_\theta f(x_j) \rangle + \epsilon(x_i)\epsilon(x_j) \langle \nabla_{W^{(2)}_{rk}} \nabla_\theta f(x_i), \nabla_\theta f(x_j) \rangle \right).$$

The first term in the sum is spanned by the same vectors as $\nabla_{W^{(2)}_{rk}} \mathcal{L}(\theta)$, and thus lives in the same subspace. Thus, this yields a data fitting contribution.

We will now compute the second term.

$$\begin{aligned}
& \sum_{ij} \epsilon(x_i)\epsilon(x_j) \langle \nabla_{W^{(2)}_{rk}} \nabla_\theta f(x_i), \nabla_\theta f(x_j) \rangle \\
= & \sum_{ij} \Big(\epsilon(x_i) \alpha^2_r(x_i) \alpha^{(1)}_k(x_j) \epsilon(x_j) \\
+ & \epsilon(x_i)\epsilon(x_j) \mathbf{1}_{[\alpha^2_r(x_i) \geq 0]} \mathbf{1}_{\left[\alpha^{(1)}_k(x_i \geq 0\right]} W^3_{1r} \cdot \langle x_i, x_j \rangle \cdot W^3_{1,:} \text{diag}(\mathbf{1}_{[a^2_:(x_j) \geq 0]}) W^{(2)}_{:,k} \textbf{vect}(\mathbf{1}_{[\alpha_k(x_j) \geq 0]}) \\
+ & \epsilon(x_i)\epsilon(x_j) \mathbf{1}_{[\alpha^2_r(x_i) \geq 0]} \mathbf{1}_{\left[\alpha^{(1)}_k(x_i) \geq 0\right]} W^3_{1r} \cdot W^3_{1,:} \text{diag}(\mathbf{1}_{[a^2_:(x_j) \geq 0]}) W^{(2)}_{:,k} \textbf{vect}(\mathbf{1}_{[\alpha_k(x_j) \geq 0]}) \Big),
\end{aligned}$$

where $\epsilon = (\epsilon(x_1), \ldots, \epsilon(x_N))^T$. This is an expression of the form $\epsilon^T M \epsilon$. Since ϵ has positive and negative entries, we essentially require M to be positive semi-definite – clearly, M with entries

$$M(x_i, x_j) = \alpha^2_r(x_i)\alpha^{(1)}_k(x_j) + (1 + \langle x_i, x_j \rangle) \cdot \mathbf{1}_{[\alpha^2_r(x_i) \geq 0]} \mathbf{1}_{\left[\alpha^{(1)}_k(x_i) \geq 0\right]} W^3_{1r} \cdot W^3_{1,:} \text{diag}(\mathbf{1}_{[a^2_:(x_j) \geq 0]}) W^{(2)}_{:,k} \textbf{vect}(\mathbf{1}_{[\alpha_k(x_j) \geq 0]})$$

depends on the entries of W^3 and the inputs x_i, x_j. This makes it intractable to guarantee that M is positive semi-definite and that the contribution to the second layer weights is non-negative. A similar computation can be made for the bias.

F.2. Computations for SGD

Assume $D = 1$. Following the regulariser given in the main text, an update is given by

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{GD}(\theta) - \frac{\eta^2}{4N} \nabla_\theta \sum_i \epsilon_i^2 \|\nabla_\theta f_\theta(x_i)\|^2.$$

It follows that

$$\nabla_\theta \sum_i \epsilon_i^2 \|\nabla_\theta f_\theta(x_i)\|^2 = 2 \sum_i \left\{ \epsilon_i \nabla_\theta f(x_i) \|\nabla_\theta f_\theta(x_i)\|^2 + \epsilon_i^2 \nabla^2_\theta f(x_i) \cdot \nabla_\theta f(x_i) \right\}.$$

We see that the first term belongs to the same subspace as the GD loss and thus constitutes a data fitting term. The second term affects the second layer weights as follows.

$$\sum_i \epsilon(x_i)^2 \nabla_{W^{(2)}_{rk}} \|\nabla_\theta f_\theta(x_i)\|^2 = \sum_i \epsilon(x_i)^2 \left\{ \alpha^2_r(x_i)\alpha^{(1)}_k(x_i) + W^3_r \mathbf{1}_{\left[\alpha^2_r(x_i) \geq 0\right]} \mathbf{1}_{\left[\alpha^{(1)}_k(x_i) \geq 0\right]} (1 + \|x_i\|^2) W_3 \text{diag}(\mathbf{1}_{\left[^{(2)}(x_i) \geq 0\right]}) W^2 \mathbf{1}_{\left[\alpha^{(1)}(x_i) \geq 0\right]} \right\}.$$

Here, we want the contributions to be non-negative, pulling down the second layer weights.